

AI Usage

The brief asks me to clearly separate AI I used to *build* this from AI the product itself *runs on*. I've tried to be straightforward about both, including how much of the build was AI-assisted, since I'd rather be accurate than make this look more hand-written than it was.

AI used to build this product

Claude (Anthropic), through the Claude.ai chat interface with its code-execution environment, was my development assistant throughout. It was used substantially: initial architecture and stack discussion, generating the backend (models, services, controllers, routes, middleware, background jobs), the React frontend, the test suite, the evaluation harness, and drafts of this documentation set. `docs/DEVELOPMENT` `PROMPTS.md` has the actual prompts, organised by area.

What I did myself, and what I'd point to as my own contribution:

- **Chose the stack and the constraints it had to work within** — React + Node/Express + MongoDB because I've built and deployed on it before, Groq because of the free tier, and the decision to attempt the full Must-Have list rather than a trimmed MVP.
- **Set up and ran everything live** — MongoDB Atlas cluster, Groq API keys, environment configuration, running both servers locally, and the Render + Vercel deployment.
- **Found the majority of the real bugs by testing the running app**, not by reading code. The retrieval tokenizer dropping "AI"/"ML"/"DL", the repetitive quiz questions, Growth being stuck on "Stable", the quiz crashing on the second question, the Tutor 500-ing on an open-ended request, the Tutor losing context when I replied to its own question, the slow "Next Question" step, the stale error message on a healthy material — each of those came from me using the product and noticing the output was wrong, then reporting it specifically enough to be diagnosed. Several of them (the tokenizer one especially) produced no error at all and would not have been caught any other way.
- **Caught that a passing test suite was lying**. The integration tests reported success while silently never executing; I pushed on why the count said "skipped" instead of accepting a green result.
- **Verified every claim in these docs**. Where something is described here as working, I ran it.

Realistically: the code volume is heavily AI-generated, and the verification, debugging direction, integration, deployment, and product judgement are mine. I've written it that way rather than dressing it up, because the brief asks for honesty here and because a reviewer comparing these docs against the commit history would see it anyway.

No other coding assistants, debugging tools, or design tools were used.

AI used by the final product

The application calls the **Groq API** for five distinct features. The model is configurable via `GROQ_MODEL` — I developed against `llama-3.3-70b-versatile` until Groq decommissioned it on the free tier mid-build, then moved to `openai/gpt-oss-120b`. Every call is logged to the `AIUsage` collection.

Feature	Where	What it does
<code>tutor</code>	<code>tutorService.askTutor</code>	Answers learner questions grounded in retrieved Project material; decides and self-reports whether evidence is sufficient; may call the <code>get_concept_mastery</code> tool.
<code>concept_extraction</code>	<code>materialProcessorService.extractConcepts</code>	Pulls 4-10 key concepts out of newly processed material, seeding <code>Concept</code> and <code>Mastery</code> records.
<code>quiz_generation</code>	<code>quizService.generateQuestion</code>	Generates one MCQ or open-ended question for a chosen concept and difficulty, grounded in retrieved material.
<code>quiz_grading</code>	<code>quizService.gradeOpenAnswer</code>	Grades open-ended answers on understanding, accuracy and relevance, returning explanatory feedback rather than only a score.
<code>recommendation</code>	<code>recommendationService.generateRecommendations</code>	Turns mastery evidence into 1-3 specific "what to do next" suggestions.

Two things deliberately **aren't** AI calls:

- **Retrieval** is TF-IDF cosine similarity computed in `retrievalService.js`. Groq has no embeddings endpoint, and I didn't want a second provider just for this.
- **Mastery updates** (`masteryService.applyEvidence`) are a deterministic confidence-weighted formula over quiz results. Mastery is the app's measurement of the learner, and I wanted it reproducible and explainable rather than dependent on a model's judgement.

Everything above funnels through one wrapper, `aiService.chatComplete`, which records model, prompt and completion tokens, latency, an estimated cost, and success or failure for every single call. That single choke point is what makes the Admin AI-usage view and per-Project AI analytics possible — if AI calls were scattered across services, none of that would exist.